

SPRINT 2 · THE GHOSTWRITER

Human in the Loop

Build an Approve / Edit / Reject Gate

MS2.3 · Thursday

"Never let an AI send anything without your approval."

-30 min · Guardrails · Streamlit UI · Approval workflows

Recap — Where We Are

Tuesday (MS2.1):

tone_profile.json — your writing personality

past_replies.json — real email examples

context_builder.py — assembles the prompt

Wednesday (MS2.2):

draft_machine.py — calls Groq API

Returns a draft reply in YOUR voice

Tested on 3 sample threads

Thursday (Today):

The missing piece: APPROVAL

The AI drafts — YOU decide

Build a UI with 3 buttons:

Approve / Edit / Reject

Context + Drafts + Approval = Safe AI assistant

Why Guardrails Matter

What happens when AI sends without approval?

Cautionary examples:

1. AI misreads sarcasm, sends a serious apology to a joke email
2. Auto-reply to boss with wrong budget numbers = career damage
3. Reply-all to 500 people with confidential info in the draft
4. Wrong tone — formal reply to a friend, casual reply to a client
5. AI hallucinates a deadline you never agreed to

The fix: a HUMAN checkpoint between 'draft' and 'send'

Rule: Any agent that takes real-world action needs a gate.

The Principle

"AI should DRAFT. Humans should DECIDE."

Levels of AI autonomy:

Level 0: AI suggests (you still do the work)

Level 1: AI drafts (you review and approve) <-- WE ARE HERE

Level 2: AI acts with constraints (send if < 50 words)

Level 3: Full autonomy (AI sends on its own) <-- DANGEROUS

The 3 Options: Approve / Edit / Reject

APPROVE

"This is good — send it as-is."

Action: Save to approved_drafts.json, mark ready to send

Use case: 60-70% of drafts if context is good

EDIT

"Close, but let me tweak it."

Action: Open text editor with draft pre-filled, then approve

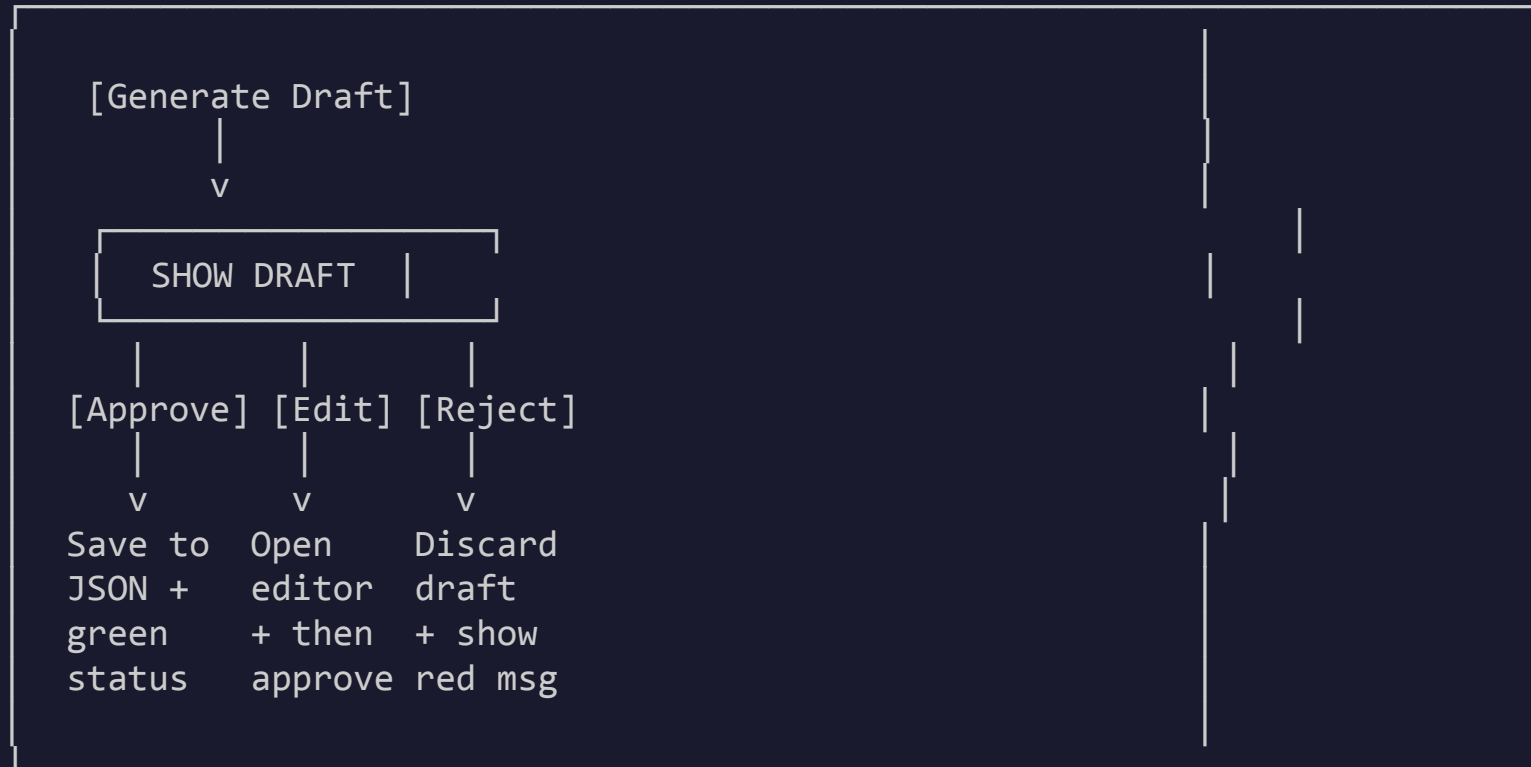
Use case: Change a date, soften a sentence, add a detail

REJECT

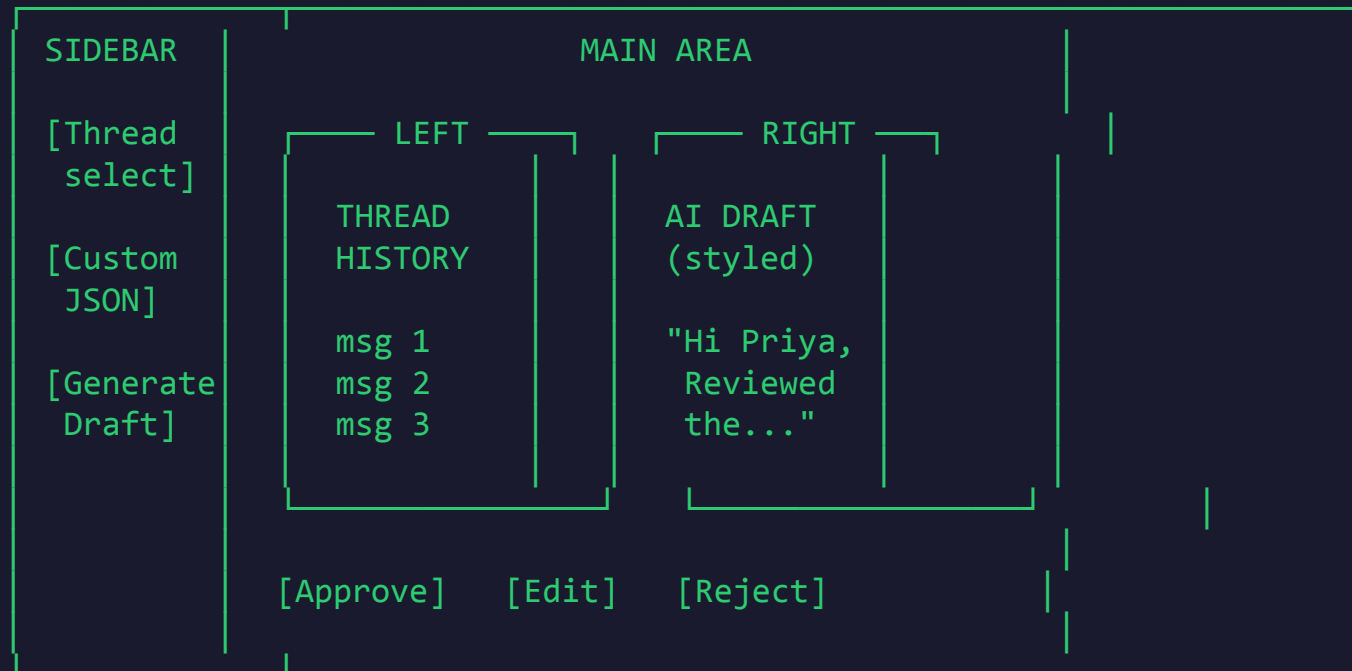
"No — completely wrong tone or content."

Action: Discard draft, option to regenerate or skip

What Happens on Each Action



The UI – Streamlit Layout



Code Structure — approval_gate.py

```
approval_gate.py
├── Imports: streamlit, context_builder, draft_machine
├── Config: page title, layout='wide'
├── Custom CSS: dark theme, boxes, status colors
├── Helper functions:
│   ├── get_api_key()          # secrets > env > input
│   ├── save_approved_draft()  # append to JSON file
│   └── render_thread()        # format thread display
├── Session state init:
│   ├── draft, status, selected_thread
│   └── edit_text, generation_count
├── Sidebar:
│   ├── Thread dropdown (SAMPLE_THREADS)
│   ├── Custom JSON text area
│   └── Generate Draft button
└── Main area:
    ├── col_thread: render thread history
    ├── col_draft: show draft + buttons
    └── Status handlers: approved/editing/rejected
```


Key Code Patterns

```
# Session state for persistence
if 'draft' not in st.session_state:
    st.session_state.draft = None
if 'status' not in st.session_state:
    st.session_state.status = None

# Two-column layout
col_left, col_right = st.columns([1, 1])

# Action buttons
c1, c2, c3 = st.columns(3)
with c1:
    if st.button('Approve'):
        save_approved_draft(draft)
        st.session_state.status = 'approved'
with c2:
    if st.button('Edit'):
        st.session_state.status = 'editing'
with c3:
    if st.button('Reject'):
        st.session_state.status = 'rejected'
```

```
# Save approved draft
def save_approved_draft(data):
    filepath = 'approved_drafts.json'
    if os.path.exists(filepath):
        approved = json.load(open(filepath))
    else:
        approved = []
    data['approved_at'] = now()
    approved.append(data)
    json.dump(approved, open(filepath, 'w'))

# API key fallback chain
def get_api_key():
    try:
        return st.secrets['GROQ_API_KEY']
    except:
        pass
    return os.environ.get('GROQ_API_KEY')

# Rerun after state change
st.rerun() # forces UI refresh
```

Let's Build It!

Build the approval gate — your AI's safety net

```
pip install streamlit groq && streamlit run approval_gate.py
```

Saturday Preview — The Draft Desk

Combine all 3 sessions into one polished tool:

MS2.1 context_builder.py — assembles your persona + thread

MS2.2 draft_machine.py — generates AI draft via Groq

MS2.3 approval_gate.py — approve / edit / reject UI

Saturday: The Draft Desk (full Streamlit app)

→ Feed in 3 real email threads

→ Generate drafts for each

→ Approve/edit/reject each one

Goal: a working prototype you'd actually use daily.

→ Export approved drafts as a batch

Today's Output

1. approval_gate.py — Streamlit app with:
 - Thread selector (dropdown + custom paste)
 - Two-column layout (thread | draft)
 - Three action buttons: Approve / Edit / Reject
 - State management via st.session_state
 - Saves approved drafts to JSON
2. Understanding of WHY guardrails matter:
 - AI drafts are not always correct
 - Human approval = safety net before real-world action
 - The approve/edit/reject pattern applies to ALL agents

Assignment

Build:

Run the Cline prompt to build `approval_gate.py`

Add your `GROQ_API_KEY`

Run: `streamlit run approval_gate.py`

Generate a draft, then try all 3 buttons

Check that `approved_drafts.json` gets created

Stretch goals:

Add a "Regenerate" button on reject

Add a confidence score to each draft

Add a "History" tab showing all past drafts

Style the approved state with confetti

Saturday: we combine everything into The Draft Desk — the full Sprint Build.

See You Saturday!

Sprint Build: "The Draft Desk" — 3 real threads, drafted and approved

Remember: AI drafts. You decide.

#MyAIChiefOfStaff